

Howard University
 College of Engineering & Architecture
 Department of Electrical Engineering & Computer Science
EECE 306-01: Fundamentals of Electromagnetics Laboratory
 Dr. Su Yan, su.yan@howard.edu

Lab 2: Coordinate Systems, Points and Vector Components

Objectives

By the end of this session you will have

- implemented the eight functions of `em.coord`,
- understood why converting a *point* and converting the *components of a vector* are two different operations,
- written a test that validates itself without any reference answer.

1. The distinction this module exists to protect

Almost every student who struggles with the next ten weeks of this subject struggles because of one confusion, so it is worth naming it on the first page.

A **point** is a location. Converting a point from Cartesian to spherical form answers the question, where is it. A **vector** is a magnitude and a direction, and it lives *at* a point. Converting the components of a vector answers a different question, namely how much of it points along $\hat{r}$, how much along $\hat{\theta}$, and how much along $\hat{\phi}$ *at that particular location*.

The reason these are different is that $\hat{r}$, $\hat{\theta}$, $\hat{\phi}$, $\hat{\rho}$ and $\hat{\phi}$ are not fixed directions. Unlike $\hat{x}$, $\hat{y}$ and $\hat{z}$, they rotate as you move around. So the components of a vector depend on where that vector sits, even when the vector itself has not changed at all.

This is why every one of the four vector conversion functions in the API takes a *second* argument giving the position.

2. Part I. Point conversion

Implement the four point functions. Each takes Nx3 and returns Nx3.

Function	Meaning
<code>em.coord.c2cyl(r)</code>	$(x, y, z) \rightarrow (\rho, \phi, z)$
<code>em.coord.cyl2c(p)</code>	$(\rho, \phi, z) \rightarrow (x, y, z)$
<code>em.coord.c2sph(r)</code>	$(x, y, z) \rightarrow (r, \theta, \phi)$
<code>em.coord.sph2c(s)</code>	$(r, \theta, \phi) \rightarrow (x, y, z)$

Conventions for this course, matching the textbook. The azimuth ϕ lies in $[0, 2\pi)$ measured from the $+x$ axis. The polar angle θ lies in $[0, \pi]$ measured from the $+z$ axis. Radians throughout.

2.1 One trap

Use `atan2`, **never** `atan`. The naive $\phi = \arctan(y/x)$ loses the quadrant, because y/x is the same for $(1, 1)$ and $(-1, -1)$, so points in opposite quadrants come out with the same azimuth. The two

argument form `atan2(y,x)` keeps the signs of both arguments and returns the correct quadrant. It returns a value in $(-\pi, \pi]$, so a wrap is needed to land in $[0, 2\pi)$.

3. Part II. Vector component conversion

Implement the four vector functions. Each takes the components and the position.

Function	Meaning
<code>em.coord.vecC2Cyl(A, r)</code>	$(A_x, A_y, A_z) \rightarrow (A_\rho, A_\phi, A_z)$ at Cartesian $\vec{r}$
<code>em.coord.vecCyl2C(Ac, r)</code>	$(A_\rho, A_\phi, A_z) \rightarrow (A_x, A_y, A_z)$ at Cartesian $\vec{r}$
<code>em.coord.vecC2Sph(A, r)</code>	$(A_x, A_y, A_z) \rightarrow (A_r, A_\theta, A_\phi)$ at Cartesian $\vec{r}$
<code>em.coord.vecSph2C(As, r)</code>	$(A_r, A_\theta, A_\phi) \rightarrow (A_x, A_y, A_z)$ at Cartesian $\vec{r}$

Note that the position argument is Cartesian in all four, including the two that *return* Cartesian components. This is deliberate. A position is a position, and keeping one canonical representation for it removes an entire class of bug, the one where a function is handed (ρ, ϕ, z) and quietly treats it as (x, y, z) .

The conversions are projections onto the local unit vectors,

$$A_\rho = \vec{A} \cdot \hat{\rho}, \quad A_\phi = \vec{A} \cdot \hat{\phi}, \quad A_r = \vec{A} \cdot \hat{r}, \quad A_\theta = \vec{A} \cdot \hat{\theta}$$

with the local unit vectors expressed in Cartesian components using the position. The conversion table in your lecture notes gives all nine entries for the spherical case.

3.1 The demonstration that makes the point

Take the vector $\vec{A} = 2\hat{x} + 3\hat{y}$, which is the same vector everywhere. Evaluate its spherical components at two different locations.

At the point $(0, 5, 0)$, the local $\hat{r}$ points along $+\hat{y}$ and the local $\hat{\phi}$ points along $-\hat{x}$, so

$$\vec{A}(0, 5, 0) = 3\hat{r} - 2\hat{\phi}$$

At the point $(4, 0, 0)$, the local $\hat{r}$ points along $+\hat{x}$ and the local $\hat{\phi}$ points along $+\hat{y}$, so

$$\vec{A}(4, 0, 0) = 2\hat{r} + 3\hat{\phi}$$

The vector did not change. Only the observer's local axes did. Reproduce both results with your code and record them in your notebook.

A consequence students get wrong on exams

Two vectors defined at *different* points cannot be added component by component in cylindrical or spherical form. $(A_r, A_\theta, A_\phi) + (B_r, B_\theta, B_\phi)$ is meaningless unless both vectors live at the same location, because the two triples are expressed in different bases. Convert both back to Cartesian, add there, and convert the sum back if you want it in curvilinear form. Your toolbox should make this easy, and your notebook should demonstrate it once.

4. Part III. The roundtrip test

Here is the star of this lab, and the first appearance of a technique used all semester.

Generate a batch of 1000 random Cartesian points. Convert them to spherical form, then convert them straight back. The result must equal the input.

```
r = randn(1000,3);
s = em.coord.c2sph(r);
r2 = em.coord.sph2c(s);
em.test.assertClose(max(em.vec.mag(r2-r)), 0, 1e-10, 'sph roundtrip');
```

Do the same for the cylindrical pair, and for both vector conversion pairs with random components at random positions.

Why this is a different kind of test

Notice what is *not* in that code. No textbook answer. No hand-worked example. No table of expected values. The test validates the code against itself, using nothing but the fact that a conversion and its inverse must compose to the identity.

This matters because for most of the problems you will meet after this course, nobody knows the answer. A verification technique that requires a known answer is useless there. A roundtrip test is not. This is the first of five reference-free techniques you will learn, and the others are symmetry, conservation laws, convergence order, and manufactured solutions.

A roundtrip test cannot catch every bug. If both directions share the same wrong sign convention, the roundtrip still closes. It is necessary, not sufficient, which is exactly why the exercise in Part II with the two positions is also required.

Required tests

Write `tests/test_lab02.m` containing at least these checks.

1. Spherical roundtrip on 1000 random points returns the input to full double precision.
2. Cylindrical roundtrip on 1000 random points, likewise.
3. Vector roundtrip, Cartesian to spherical components and back, at 1000 random positions with 1000 random component triples.
4. Vector roundtrip for the cylindrical pair.
5. `em.coord.c2sph([0 0 1])` returns $r = 1$ and $\theta = 0$, and `em.coord.c2sph([0 0 -1])` returns $\theta = \pi$.
6. ϕ lies in $[0, 2\pi)$ for points in all four quadrants of the xy plane.
7. The magnitude of a vector is unchanged by any component conversion. Check on 1000 random cases. This is a second reference-free invariant and it catches errors the roundtrip misses.
8. The two worked cases from Part II, $(0, 5, 0)$ and $(4, 0, 0)$.

`runTests` must report Lab 1 and Lab 2 both passing. If a Lab 1 test has broken, find out why before you leave.

Deliverables

Submit through Canvas by the start of the next lab session.

1. **Toolbox snapshot**, `TeamNN_Lab02.zip`.

2. **Lab notebook**, published script exported to PDF, three to five pages. It must contain the two-position demonstration from Part II with your computed numbers, and the `runTests` output.
3. **Test output** pasted into the notebook.

Looking ahead

Next week the toolbox starts computing physics. You will build the source struct and the field engine, and you will see the architectural decision that makes every later lab cheaper. Read Section 2.2 of Wentworth before you come.